



SMART KITCHEN CONTROL SYSTEM

FreeRTOS-Based Smart Lighting and Oven Safety Controller Advanced Embedded Systems

Course Code: CSE323/CSE411

Summer Semester 2026

Project Team (10)

Responsibility	Student Name	Student ID
Role 1 - Lighting System Developer	Ali Ibrahim	23P0217
Role 2 - Oven System & Safety Developer	Kareem Ahmed Talat	23P0225
Role 3 - RTOS Architecture & Synchronization	Zakaria Eissa	22P0128
Role 4 - Hardware Integration & Faults	Ferass Ahmed Moustafa	23P0304
Role 5 - Technical Documentation & Demo	Ahmad Hatem	23P0322

Date of Submission: 15 August 2026

Demo Video Link: [Click Here](#)

TABLE OF CONTENTS

Abstract	4
1. Introduction	4
1.1 Background	4
1.2 Project Objectives	4
1.3 Scope and Submitted Deliverables	4
2. System Requirements and Design Overview	5
2.1 Functional Requirements	5
2.2 Inputs and Outputs	5
2.3 Safety Precedence	6
3. Hardware Platform, Components and Wiring	6
3.1 Development Platform	6
3.2 Components Used	6
3.3 Pin Mapping	7
3.4 Wiring Diagram	7
4. Software Architecture and RTOS Design	8
4.1 Task Model and Priorities	8
4.2 Scheduler and Memory Configuration	8
4.3 Inter-Task Communication and Synchronization	8
4.4 Overall Architecture	9
5. Program Flowcharts	10
5.1 Main Program Flow	10
5.2 Lighting Control Task Flow	11
5.3 Oven Control Task Flow	12
5.4 User Override Task Flow	13
5.5 UART Logging Task Flow	13
6. Fault Handling and Safety Strategy	14
6.1 Why Safety Overrides Manual Control	14
7. Testing and Demonstration Procedure	15
7.1 PuTTY Setup	15
8. Problems Faced and Solutions	16
8.1 Remaining Limitations	16
9. Detailed Software Design & Prototype Photos	17
9.1 main.c - Initialization and Scheduler Start	17
9.2 tasks.c / tasks.h - Shared RTOS Layer	17

9.3 Task 1 - Lighting Control	17
9.4 Task 2 - Oven Control and Safety	18
9.5 Task 3 - User Override	18
9.6 Task 4 - UART Logging	19
9.7 adc.c / adc.h - Sensor Interface.....	19
9.8 switch.c / switch.h - Digital Inputs and ISRs.....	19
9.9 led.c / led.h - Actuator Driver	19
9.10 uart.c / uart.h - Serial Driver.....	19
10.11 Prototype Photos and Demo Video.....	20
10.12 Demo Video Link	20
11. Project Evaluation Mapping and Lessons Learned.....	21
11.1 Lessons Learned.....	21
12. Conclusion and Recommendations for Future Work	21
12.1 Conclusion	21
12.2 Recommendations for Future Work.....	21
References	22

Abstract

This report presents the design and implementation of a real-time Smart Kitchen Control System developed for CSE323/CSE411: Advanced Embedded Systems. The system combines a smart lighting subsystem, an oven temperature control and safety subsystem, manual user overrides, hardware fault handling, and serial monitoring over UART. FreeRTOS is used to execute four concurrent tasks with preemptive scheduling and time slicing: Lighting Control and Oven Control at equal normal priority, User Override at the highest application priority, and UART Logging at a lower priority. Queues, mutexes, and a counting semaphore are used for inter-task communication and synchronization. The final implementation targets the Texas Instruments TM4C123GH6PM Cortex-M4 microcontroller and uses onboard LEDs and switches together with ADC-based temperature sensing and bench-demo jumper inputs. Safety is deliberately prioritized over convenience: an invalid temperature reading or a critical over-temperature condition forces the oven element OFF even in Manual mode, clears any latched manual oven request, and produces an immediate UART alert. The project demonstrates practical RTOS scheduling, synchronization, resource protection, hardware interfacing, fault containment, and team-based embedded development.

1. Introduction

1.1 Background

Modern smart-home systems rely on embedded controllers that must react predictably to sensor readings, user inputs, and safety events. A kitchen is an especially suitable real-time application because lighting decisions are periodic and convenience-oriented, whereas oven temperature faults require immediate and deterministic handling. The major task therefore requires a FreeRTOS application that integrates smart lighting, smart oven control, user override inputs, and UART-based monitoring in one concurrent system.

1.2 Project Objectives

- Implement a multitasking FreeRTOS application on an ARM Cortex-M4 platform.
- Use preemptive scheduling and equal-priority tasks so round-robin time slicing can be observed when tasks are simultaneously ready.
- Monitor ambient light and oven temperature and automatically control their corresponding actuators.
- Provide low-latency user overrides without allowing a manual oven command to defeat critical safety cut-offs.
- Use queues, mutexes, and a counting semaphore for safe inter-task communication and resource protection.
- Detect sensor and switch fault conditions and place the system in a safe state.
- Report system state and critical events through UART to a PuTTY console.

1.3 Scope and Submitted Deliverables

The report is structured to match the major-task deliverables. It contains the technical design, detailed code description, selected code excerpts placed beside the relevant explanations, main program flowcharts, hardware component list and wiring, team contribution table, prototype-photo placeholders, problems and solutions, conclusions, future recommendations, and references. The complete C and header source files remain a separate source-code deliverable in the project archive, while only the most relevant excerpts are shown in this report to explain key design decisions.

2. System Requirements and Design Overview

2.1 Functional Requirements

ID	Area	Requirement	Implementation
FR-01	Lighting	Sample ambient light and turn the kitchen light ON in darkness and OFF in brightness during Auto mode.	Implemented in Tasks_LightingControl().
FR-02	Lighting	Allow a manual lighting toggle that takes precedence over automatic lighting logic in Manual mode.	PF4/SW1 event handled by the Override task.
FR-03	Oven	Sample oven temperature and control the heating element automatically.	Internal temperature sensor, 500 ms sampling.
FR-04	Oven Safety	Force the heating element OFF at the critical threshold or if the temperature reading is invalid.	30.0 C bench critical threshold; invalid ADC/fault jumper forces OFF.
FR-05	Manual Override	React to user override events at the highest application priority.	Override task priority = 3.
FR-06	Monitoring	Transmit state changes, telemetry and critical alerts to PuTTY over UART.	UART0 at 9600 baud, 8N1.
FR-07	Synchronization	Use RTOS queues, mutexes and a counting semaphore.	Two queues, two mutexes and one counting semaphore.
FR-08	Fault Handling	Detect sensor disconnect/out-of-range states and abnormal switch activity.	Sensor fault latching, bounded ADC wait, debounce and stuck-switch detection.

2.2 Inputs and Outputs

Type	Signal	Interface	Purpose
Input	Ambient light	PE5 demo jumper; real LDR path prepared on PE3/AIN0	Dark/bright decision
Input	Oven temperature	TM4C123 internal temperature sensor via ADC0 SS3	Automatic heat control and safety
Input	Mode selector	PB0 with pull-up	Auto/Manual mode
Input	Light manual toggle	PF4 / onboard SW1	Manual light request
Input	Oven manual toggle	PF0 / onboard SW2	Manual oven request
Input	Fault simulation	PE1 with pull-up; jumper to GND	Simulated temperature sensor disconnect
Output	Kitchen light actuator	PF3 / onboard green LED	Lighting state
Output	Oven heating actuator	PF1 / onboard red LED	Heating element state
Output	Serial monitoring	UART0 TX on PA1 through USB debug	PuTTY logs and telemetry

2.3 Safety Precedence

The oven controller uses an explicit safety precedence. Sensor validity is checked first, then the critical temperature threshold, then the over-temperature hysteresis latch, and only after these safety checks is Manual or Auto control allowed to determine the requested heater state. Therefore, Manual mode does not bypass thermal safety. This is a deliberate fail-safe design choice.

3. Hardware Platform, Components and Wiring

3.1 Development Platform

The Keil project targets the Texas Instruments TM4C123GH6PM microcontroller, a 32-bit ARM Cortex-M4 device. The project uses the Keil TM4C Device Family Pack and the ARM CMSIS-FreeRTOS 11.3.0 pack. Application peripheral drivers are written directly against memory-mapped TM4C registers, which keeps the project independent of an external DriverLib library.

3.2 Components Used

No.	Component	Use in Project
1	TM4C123G/TM4C123GH6PM Tiva C LaunchPad	Main controller, GPIO, ADC, UART, internal temperature sensor
2	Onboard green LED (PF3)	Kitchen lighting actuator indication
3	Onboard red LED (PF1)	Oven heating-element indication
4	Onboard SW1 (PF4)	Manual light toggle
5	Onboard SW2 (PF0)	Manual oven toggle
6	Jumper wire on PB0	Auto/Manual mode selector
7	Jumper wire on PE1	Temperature-sensor disconnect simulation
8	Jumper wire on PE5	Active bench light-sensor simulation
9	LDR (real-sensor option)	Voltage divider to PE3/AIN0; code path is present but currently disabled
10	USB debug cable and PC	Power/programming and UART link to PuTTY
11	PuTTY terminal	Displays real-time system logs at 9600 baud, 8 data bits, no parity, 1 stop bit

3.3 Pin Mapping

Pin	Configuration	Function	Notes
PE5	GPIO input	Bench light simulation	LOW/GND -> 3000 (dark); HIGH/open -> 500 (bright)
PE3 / AIN0	ADC0 SS2	Real LDR option	Prepared analog path; currently disabled in adc.c
Internal TS	ADC0 SS3	Temperature sensor	Converted to tenths of degrees Celsius
PE1	GPIO input + pull-up	Fault simulator	GND = valid; open = invalid temperature
PB0	GPIO input + pull-up	Mode select	GND = AUTO; open = MANUAL
PF4	GPIO input + interrupt	SW1 light toggle	Falling-edge interrupt
PF0	GPIO input + interrupt	SW2 oven toggle	Falling-edge interrupt
PF3	GPIO output	Green LED	Kitchen light
PF1	GPIO output	Red LED	Oven heating element
PA0/PA1	UART0 RX/TX	Serial terminal	TX used for logs; 9600 baud

3.4 Wiring Diagram

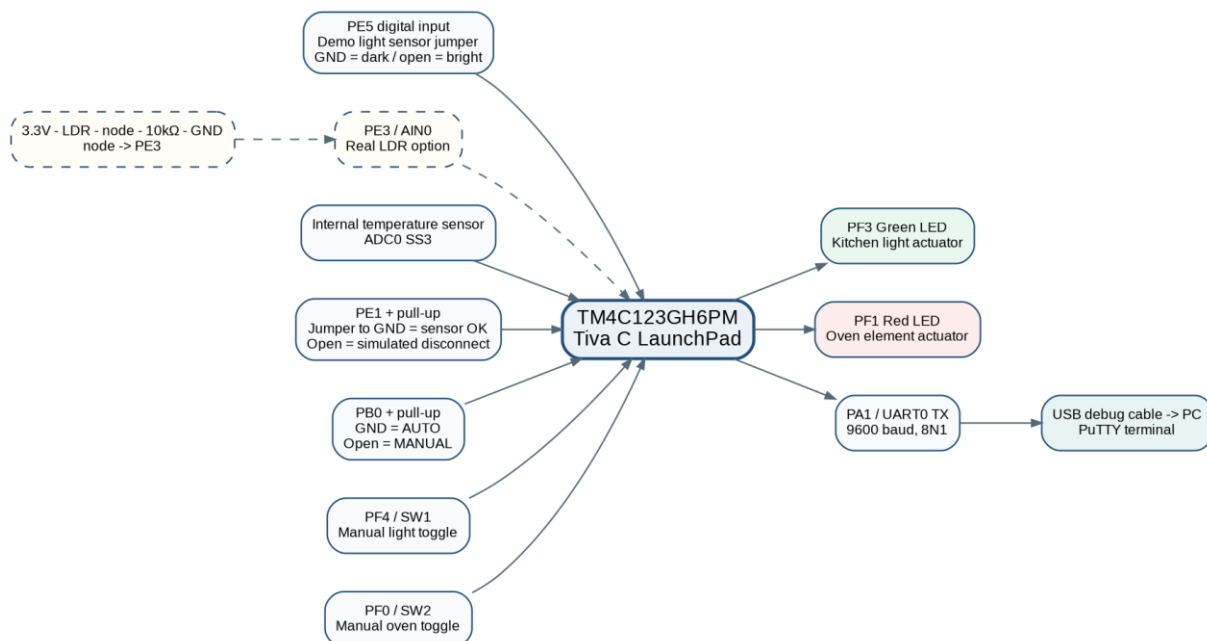


Figure 1. Smart Kitchen hardware pin mapping and bench-demo wiring.

4. Software Architecture and RTOS Design

4.1 Task Model and Priorities

Lighting Control and Oven Control deliberately share priority 2. FreeRTOS is configured for preemption and time slicing, so if both equal-priority tasks are Ready at the same time they can round-robin across tick boundaries. The Override task has higher priority so switch events can preempt the control tasks. The UART task has lower priority because serial reporting is less time-critical than control and safety decisions.

Task	Priority	Activation	Main Responsibility
Task 3 - User Override	3 (highest application)	Event-driven; blocks on counting semaphore	Updates mode/manual requests with low latency
Task 1 - Lighting Control	2	Periodic, 500 ms	Samples light and controls PF3
Task 2 - Oven Control	2	Periodic, 500 ms	Samples temperature, applies safety, controls PF1
Task 4 - UART Logging	1	Event-driven; blocks on log queue	Serializes routine log output to PuTTY

4.2 Scheduler and Memory Configuration

On a Cortex-M4 where StackType_t is 32 bits, a task stack depth of 256 entries is approximately 1 KiB per task. Four application tasks therefore reserve roughly 4 KiB for task stacks, before task-control blocks, queues, semaphores and other kernel structures. The 8 KiB configured heap provides a practical margin for this small application. The stack-overflow hook in main.c disables interrupts and enters a safe infinite loop if a stack overflow is detected.

4.3 Inter-Task Communication and Synchronization

Object	Type / Capacity	Users	Reason
xLogQueue	Queue, 10 x LogMessage_t (80 chars)	Control/override tasks -> UART task	Decouples control timing from blocking serial transmission
xOverrideQueue	Queue, 10 x ButtonEvent_t	GPIO ISRs -> Override task	Carries mode/light/oven button events
xOverrideSemaphore	Counting semaphore, max 10	GPIO ISRs -> Override task	Wakes the high-priority handler once for each queued event
xStateMutex	Mutex	Override task + control tasks	Protects global mode and manual request flags
xUARTMutex	Mutex	UART Logging + critical oven path	Prevents concurrent UART string corruption

4.4 Overall Architecture

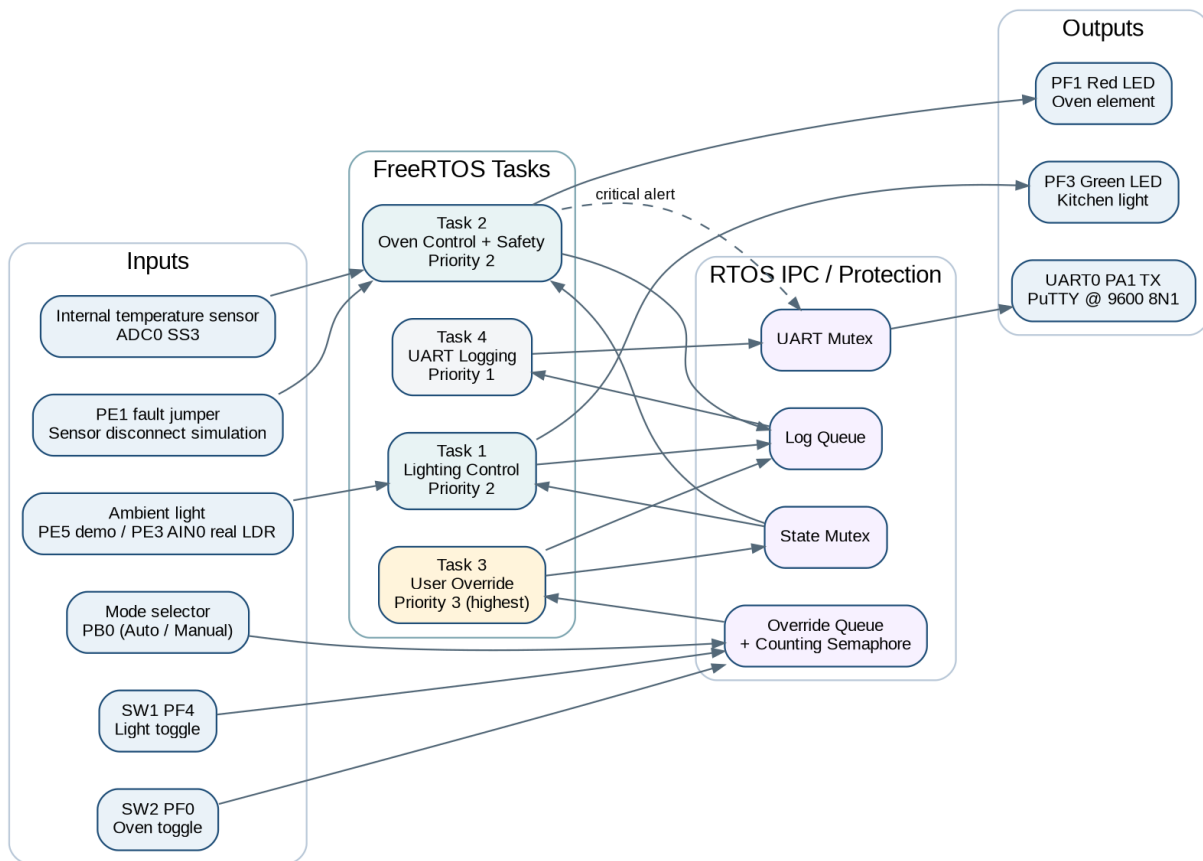


Figure 2. FreeRTOS task, input/output and synchronization architecture.

5. Program Flowcharts

5.1 Main Program Flow

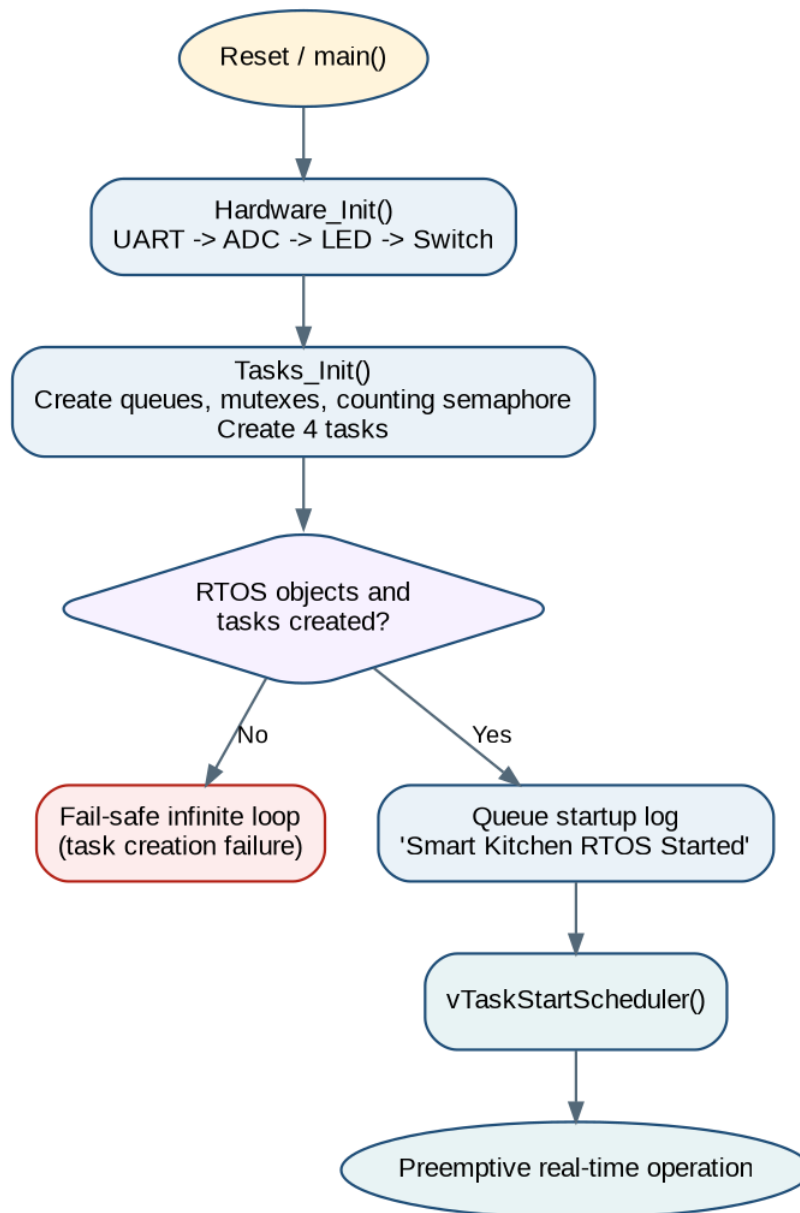


Figure 3. Main initialization and FreeRTOS scheduler flow.

5.2 Lighting Control Task Flow

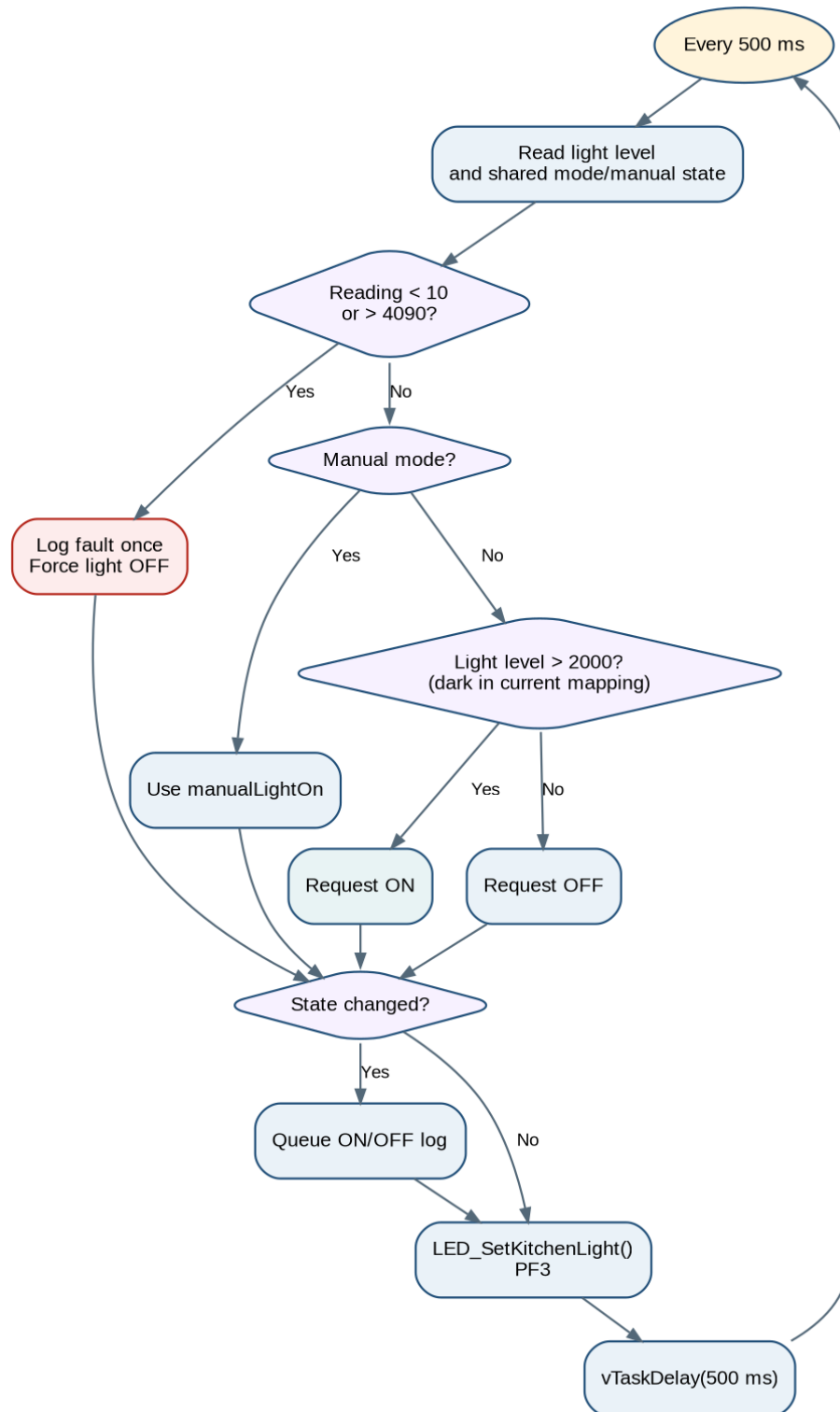


Figure 4. Lighting Control task decision flow.

5.3 Oven Control Task Flow

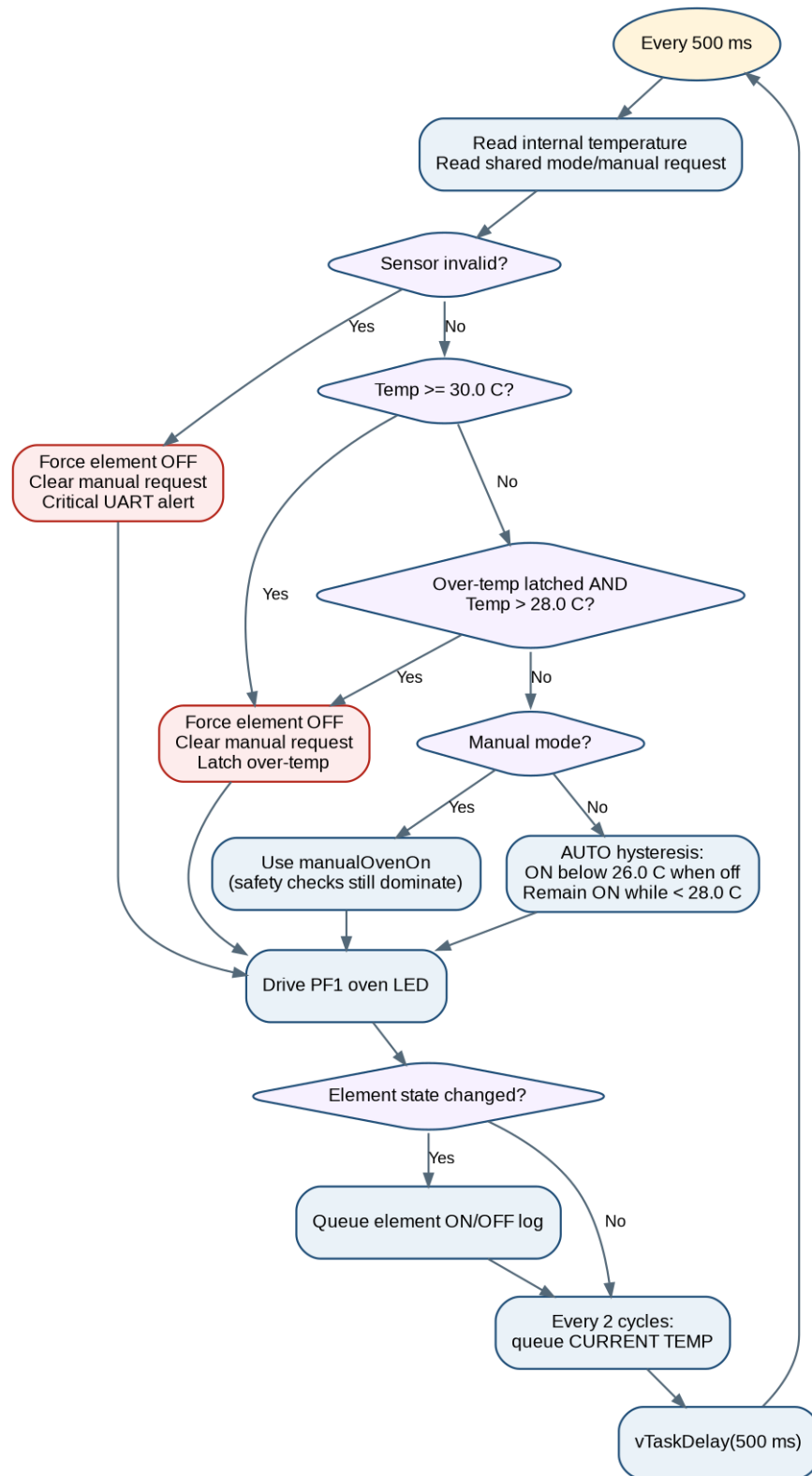


Figure 5. Oven Control task with fail-safe precedence and hysteresis.

5.4 User Override Task Flow

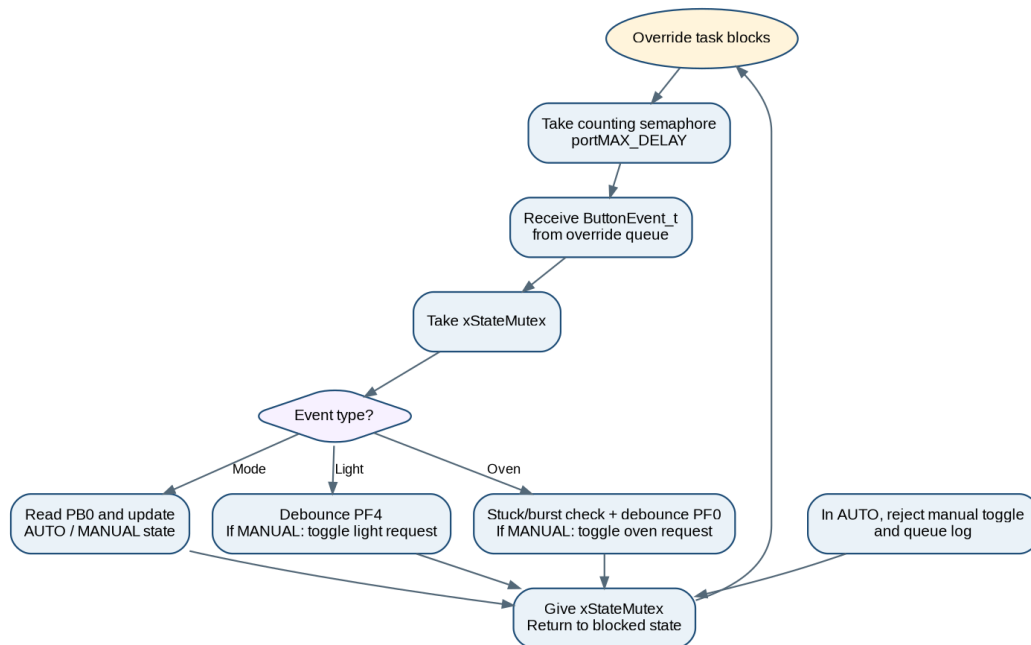


Figure 6. High-priority manual override event handling.

5.5 UART Logging Task Flow

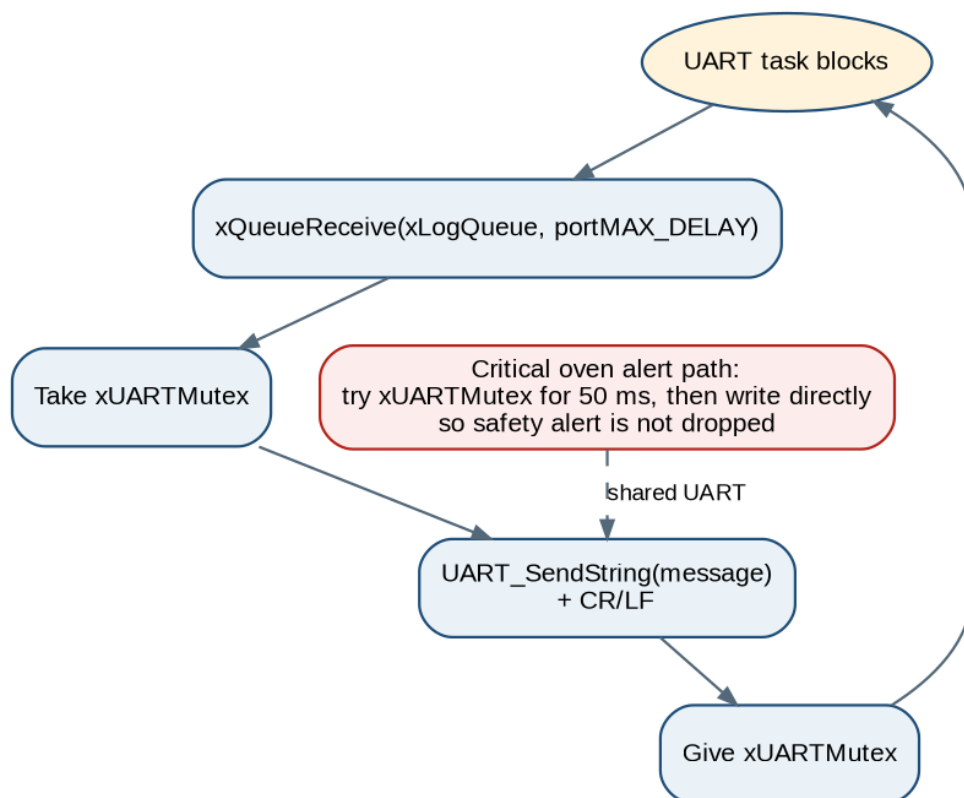


Figure 7. Queue-based UART logging and critical-alert path.

6. Fault Handling and Safety Strategy

6.1 Why Safety Overrides Manual Control

The project brief requires both manual oven control and thermal safety cut-off. If interpreted naively, these requirements can conflict. The final implementation resolves the conflict by giving sensor validity and critical temperature higher precedence than the manual command. This prevents a user from holding the heating element ON while the measured temperature is unsafe or unknown. The manual request is also cleared during a fault so the element cannot restart automatically when the fault disappears.

Fault / Hazard	Detection	Safe Response
Temperature sensor disconnect / invalid ADC	PE1 open or ADC conversion timeout returns ADC_TEMP_TENTHS_INVALID	Element forced OFF; manual oven request cleared; critical UART alert
Over-temperature	Temperature ≥ 30.0 C bench threshold	Element forced OFF; over-temp latched; manual request cleared; critical temperature logged
Temperature near critical recovery band	Over-temp active and temperature still > 28.0 C	Element stays OFF until safely below the band
Light sensor rail/disconnect	Reading < 10 or > 4090 on real ADC path	Light forced OFF; fault logged once; recovery logged once
Oven override switch burst/stuck behavior	At least 5 events with intervals inside the 30-tick window	Event ignored and fault message queued
Mechanical switch bounce	Event within 50 ms or pin no longer physically pressed	Event ignored
ADC peripheral never finishes	Bounded wait counter expires	Temperature treated invalid instead of hanging control task
UART queue saturation during safety event	Critical alert does not depend on xLogQueue	Direct protected/best-effort UART write
Task stack overflow	FreeRTOS stack checking level 2 calls application hook	Interrupts disabled; system trapped for debugging rather than continuing corrupted execution

7. Testing and Demonstration Procedure

The following test matrix is suitable for the final bench demonstration and 5-minute video. The Expected Result column is derived directly from the implemented logic; the team should capture the corresponding LED/PuTTY evidence during the final hardware run.

ID	Test	Procedure	Expected Result
T1	Boot system	Power/reset LaunchPad	Startup message appears; initial actuators OFF; scheduler starts
T2	Auto light - dark	PB0 to GND (AUTO); PE5 to GND	Green PF3 LED turns ON; KITCHEN LIGHT: ON logged
T3	Auto light - bright	PB0 AUTO; PE5 open	Green PF3 LED turns OFF; state change logged
T4	Manual light	PB0 open (MANUAL); press SW1	Each valid press toggles green LED and manual-light log
T5	Auto oven below setpoint	PE1 to GND; temperature below auto ON threshold	Red PF1 heating LED commanded ON; temperature logged once per second
T6	Critical temperature	Warm sensor until ≥ 30.0 C	Red PF1 forced OFF; CRITICAL over-temperature alert appears
T7	Manual oven safety	Manual mode, request oven ON, then cross critical threshold	Safety cut-off still forces oven OFF and clears manual request
T8	Temperature disconnect	Pull PE1 jumper	Immediate FAULT message; red PF1 forced OFF
T9	Mode change	Insert/remove PB0 jumper	SYSTEM: AUTO MODE / MANUAL MODE logged
T10	Rapid oven switch edges	Generate abnormal repeated PF0 events	Burst is rejected with oven-switch fault message
T11	UART serialization	Cause several state updates close together	Readable complete lines on PuTTY; no routine-message corruption

7.1 PuTTY Setup

- Connect the LaunchPad USB debug cable to the PC.
- Open the correct COM port in PuTTY.
- Select Serial mode, 9600 baud, 8 data bits, no parity and 1 stop bit.
- Reset the board and verify the startup message before performing the test matrix.

8. Problems Faced and Solutions

Problem	Cause / Risk	Resolution
Duplicate task/source definitions during integration	Role-separated files and project entries caused duplicate functions/object files.	Final task implementations were consolidated in tasks.c and only one project entry is retained for each application source file.
Undefined vApplicationStackOverflowHook at link time	Stack overflow checking was enabled but the application hook was missing.	Implemented vApplicationStackOverflowHook() in main.c outside main().
FreeRTOS timer-daemon linker symbols	Software timers were enabled without the timer module being required/linked.	Set configUSE_TIMERS to 0 because this application uses vTaskDelay and no software timers.
UART symbol / logging integration mismatch	Different task code expected inconsistent UART helper names or duplicate logger implementations.	Standardized the driver interface around UART_SendString() and one UART Logging task.
No 10 kΩ resistor for the LDR voltage divider	The real PE3/AIN0 hardware could not be assembled for the bench demo.	Kept the real ADC code and added a PE5 jumper simulation that returns safe, deterministic dark/bright values.
Manual oven request could conflict with safety	A direct manual command could otherwise keep the element ON at unsafe temperature.	Reordered logic so invalid-sensor and critical-temperature checks always execute before Manual/Auto control and clear the manual latch on fault.
Heater output chattering near threshold	Small sensor changes around the setpoint could repeatedly toggle the element.	Added a 2.0 C hysteresis band.
Mechanical switch bounce / stuck switch	GPIO edges can occur much faster than intentional user presses.	Added ISR/task debounce plus an oven burst detector and ignored abnormal events.
Potential task hang on ADC conversion	An unbounded ADC polling loop can permanently block a control task if hardware fails.	Added ADC_WAIT_LIMIT and return an invalid sensor value on timeout.
Concurrent UART writers	Critical safety logging and queued routine logging can overlap.	Protected UART with xUARTMutex while allowing a safety-biased direct alert fallback if the mutex is not acquired quickly.

8.1 Remaining Limitations

- The current light sensor uses a digital PE5 simulation rather than the real LDR/ADC path. The real LDR should be restored before a final physical-sensor demonstration if the required resistor is available.
- The internal MCU temperature sensor is appropriate for a classroom bench demonstration but is not a real oven probe. A thermocouple/RTD or suitable external temperature IC would be required for practical oven deployment.
- The GPIO handlers include a short busy-wait delay for hardware debounce. A production RTOS design should keep ISRs shorter and perform debounce in a task or timer-based state machine.
- The UART driver is polling-based. DMA or interrupt-driven transmission could reduce CPU blocking for larger logs.
- A strict four-physical-output requirement would benefit from an added buzzer or dedicated fault indicator.

9. Detailed Software Design & Prototype Photos

9.1 main.c - Initialization and Scheduler Start

main.c is intentionally small. Hardware_Init() initializes UART, ADC, LED and switch drivers. Tasks_Init() then creates the two queues, two mutexes, counting semaphore and all four application tasks. If any RTOS object or task creation fails, main enters an infinite fail-safe loop rather than starting a partially initialized system. A startup message is queued before vTaskStartScheduler() transfers execution to FreeRTOS. The file also defines vApplicationStackOverflowHook(), required because stack-overflow checking is enabled in FreeRTOSConfig.h.

The entry point demonstrates the intended separation between hardware initialization, RTOS object/task creation, and scheduler start:

```
int main(void)
{
    Hardware_Init();
    if (Tasks_Init() != pdPASS) { while (1) {} }
    Tasks_LogSend("Smart Kitchen RTOS Started", 0);
    vTaskStartScheduler();
}
```

9.2 tasks.c / tasks.h - Shared RTOS Layer

tasks.h defines task priorities, queue sizes, message structures, system-mode enumerations and oven safety constants. tasks.c owns all RTOS objects and shared state. This consolidation avoids duplicate task definitions during integration and provides one clear synchronization boundary for mode and manual actuator requests.

The shared RTOS layer creates the queues, mutexes, counting semaphore, and four tasks in one place. The two control tasks deliberately use the same priority:

```
xLogQueue = xQueueCreate(LOG_QUEUE_LENGTH, sizeof(LogMessage_t));
xOverrideQueue = xQueueCreate(OVERRIDE_QUEUE_LENGTH, sizeof(ButtonEvent_t));
xStateMutex = xSemaphoreCreateMutex();
xUARTMutex = xSemaphoreCreateMutex();
xOverrideSemaphore = xSemaphoreCreateCounting(OVERRIDE_QUEUE_LENGTH, 0);

xTaskCreate(Tasks_LightingControl, "LightCtrl", configMINIMAL_STACK_SIZE * 2,
            NULL, CONTROL_TASK_PRIORITY, NULL);
xTaskCreate(Tasks_OvenControl, "OvenCtrl", configMINIMAL_STACK_SIZE * 2,
            NULL, CONTROL_TASK_PRIORITY, NULL);
```

9.3 Task 1 - Lighting Control

Tasks_LightingControl() executes every 500 ms. It reads the light sensor and snapshots the protected system state. A reading below 10 or above 4090 is treated as a disconnected/rail fault for the real ADC path and forces the light OFF. In Manual mode the task follows manualLightOn. In Auto mode the current demo mapping treats values above 2000 as dark, which requests the light ON. State changes are logged only when the commanded output changes, reducing repetitive UART traffic. PF3 is then updated through LED_SetKitchenLight().

The core lighting decision is compact: a sensor fault forces the light OFF, Manual mode uses the stored override, and Auto mode compares the reading with the threshold:

```
if (lightLevel > 4090 || lightLevel < 10) {
    turnOn = false;
} else if (mode == SYSTEM_MODE_MANUAL) {
    turnOn = lightManual;
} else {
    turnOn = (lightLevel > LIGHT_THRESHOLD);
}
LED_SetKitchenLight(turnOn);
```

9.4 Task 2 - Oven Control and Safety

Tasks_OvenControl() is the principal safety task. It samples the internal temperature sensor every 500 ms and represents temperature in tenths of a degree Celsius so comparisons remain integer-based. The bench setpoint is 28.0 C, the critical cut-off is 30.0 C, and the hysteresis band is 2.0 C. These low values are deliberately chosen so the internal sensor can be influenced during a classroom demonstration; a real oven would require a calibrated external thermal sensor and realistic setpoints.

The decision order is fail-safe. An invalid temperature immediately forces PF1 OFF, clears the manual oven request and emits a critical alert. A reading at or above 30.0 C also forces the element OFF and latches the over-temperature state. Heating cannot resume until temperature falls below the critical release band. In Manual mode the requested state comes from manualOvenOn only after all safety checks pass. In Auto mode hysteresis prevents rapid element chattering around the setpoint. Temperature telemetry is queued once per second.

The following excerpt shows the safety precedence. A faulty temperature sensor or a critical temperature can never be overridden by the manual oven request:

```
if (sensorFault) {
    turnOn = false;
    Tasks_ClearOvenManual();
}
else if (tempTenths >= OVEN_CRITICAL_TENTHS) {
    turnOn = false;
    Tasks_ClearOvenManual();
}
else if (mode == SYSTEM_MODE_MANUAL) {
    turnOn = ovenManual;
}
```

Temperature conversion: The internal TM4C123 sensor is converted using the integer form $T(0.1\text{ C}) = 1475 - (2475 \times \text{ADC}) / 4096$. The sensor has a negative ADC slope, so higher ADC code corresponds to lower temperature.

9.5 Task 3 - User Override

GPIO interrupt handlers never perform the full control decision. They post a ButtonEvent_t to xOverrideQueue, give xOverrideSemaphore and request a context switch with portYIELD_FROM_ISR() if the event wakes the higher-priority task. Tasks_UserOverride() blocks on the counting semaphore, receives one queued event, takes xStateMutex and updates the shared system state. Mode changes read PB0; PF4 toggles the manual light request; PF0 toggles the manual oven request. Manual toggles are rejected in Auto mode. A 50 ms software cooldown and physical-state check reduce switch bounce, while a separate burst counter flags oven override activity that is too fast to be a normal human press.

Instead, the ISR posts a short event and wakes the high-priority override task. This keeps the interrupt path focused on notification rather than application logic:

```
if (xQueueSendFromISR(xOverrideQueue, &event,
    &xHigherPriorityTaskWoken) == pdPASS) {
    xSemaphoreGiveFromISR(xOverrideSemaphore,
    &xHigherPriorityTaskWoken);
}
portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
```

9.6 Task 4 - UART Logging

Tasks_UARTLogging() blocks on xLogQueue, so it consumes no CPU while no messages are pending. When a message arrives it acquires xUARTMutex, transmits the message and CR/LF terminator, then releases the mutex. Routine status reporting therefore cannot corrupt another UART write. Safety-critical oven alerts use Tasks_LogCritical(), which tries to acquire the same UART mutex for 50 ms and then writes directly even if the mutex is unavailable; the design intentionally prefers a potentially interleaved critical alert over a silent safety event.

The logging task blocks until a message is available and protects the UART peripheral with a mutex before transmission:

```
if (xQueueReceive(xLogQueue, &receivedMessage, portMAX_DELAY) == pdPASS) {
    if (xSemaphoreTake(xUARTMutex, portMAX_DELAY) == pdTRUE) {
        UART_SendString(receivedMessage.text);
        UART_SendString("\r\n");
        xSemaphoreGive(xUARTMutex);
    }
}
```

9.7 adc.c / adc.h - Sensor Interface

ADC_Init() enables ADC0 and GPIOE, configures PE3 for the optional LDR input, PE1 as a pulled-up fault input, and PE5 as the current light-sensor demo input. ADC0 sample sequencer 3 is configured for the internal temperature sensor and sample sequencer 2 is prepared for the real LDR.

ADC_WaitForConversion() uses a bounded polling loop instead of an infinite wait; if conversion never completes, the function returns an invalid reading and the oven task enters its safe OFF state. The active light function returns 3000 when PE5 is grounded and 500 when open, providing deterministic dark/bright test values.

9.8 switch.c / switch.h - Digital Inputs and ISRs

Switch_Init() unlocks PF0, configures PF0/PF4 with pull-ups and falling-edge interrupts, and configures PB0 with a pull-up and both-edge interrupt for mode insertion/removal. GPIOF_Handler() posts light or oven toggle events, while GPIOB_Handler() posts a mode-update event. Interrupt priority 5 is chosen so the handlers may safely call the FreeRTOS FromISR APIs under the configured Cortex-M interrupt-priority rules.

9.9 led.c / led.h - Actuator Driver

The LED driver configures PF1 and PF3 as digital outputs without disturbing the Port F switch pins. LED_SetKitchenLight() controls PF3 and LED_SetOvenElement() controls PF1 using atomic-style bit set/clear operations on the GPIO data register. Both outputs are explicitly initialized OFF.

9.10 uart.c / uart.h - Serial Driver

UART_Init() configures UART0 on PA0/PA1 at 9600 baud for a 16 MHz UART clock assumption. The line format is 8 data bits, no parity and 1 stop bit. UART_WriteChar() polls the transmit FIFO-full flag, while UART_SendString() sends a null-terminated string character by character. RX interrupts are deliberately disabled because this project only transmits telemetry; enabling an RX interrupt without a UART0_Handler would cause an exception.

The UART baud-rate registers are configured for 9600 baud under the project's 16 MHz clock assumption:

```
UART0_IBRD_R = 104;
UART0_FBRD_R = 11;
UART0_LCRH_R = 0x60; /* 8 data bits, no parity, 1 stop bit */
UART0_CTL_R = 0x301; /* Enable UART, TX and RX */
```

10.11 Prototype Photos and Demo Video



Figure 8. Final assembled Smart Kitchen prototype showing the LaunchPad and jumper connections.

```

CURRENT TEMP: 30.9 C
KITCHEN LIGHT: ON
CURRENT TEMP: 30.3 C
CURRENT TEMP: 29.4 C
CURRENT TEMP: 30.0 C
CURRENT TEMP: 29.6 C
CURRENT TEMP: 29.6 C
CURRENT TEMP: 29.4 C
CURRENT TEMP: 29.5 C
CURRENT TEMP: 29.7 C
CURRENT TEMP: 28.7 C
CURRENT TEMP: 29.2 C
CURRENT TEMP: 29.6 C
OVEN ELEMENT: ON
CURRENT TEMP: 19.9 C
CURRENT TEMP: 29.5 C
CURRENT TEMP: 28.9 C
CURRENT TEMP: 29.5 C
CURRENT TEMP: 29.8 C
CURRENT TEMP: 28.7 C
CURRENT TEMP: 29.7 C
OVEN ELEMENT: OFF
CURRENT TEMP: 30.5 C
KITCHEN LIGHT: OFF
CURRENT TEMP: 31.2 C
CRITICAL: OVEN OVER TEMPERATURE - ELEMENT F
CURRENT TEMP: 41.9 C
CURRENT TEMP: 40.9 C
CURRENT TEMP: 41.8 C
  
```

Figure 9. PuTTY console showing startup, mode changes, actuator states and temperature telemetry.

10.12 Demo Video Link

Final video URL: [Click here](#)

11. Project Evaluation Mapping and Lessons Learned

11.1 Lessons Learned

- Priority design should reflect urgency: user-event handling can be high priority while slow UART output remains low priority.
- Queues are effective for decoupling producers from a slow shared output device such as UART.
- A mutex is appropriate for a shared hardware peripheral, while a counting semaphore is useful when multiple ISR events may accumulate.
- Safety requirements must be assigned explicit precedence when they conflict with manual convenience features.
- Hysteresis, bounded waits and debounce logic are small implementation details that significantly improve real-time robustness.
- Keeping drivers separate from RTOS task logic made the five-person team split and final integration easier to manage.

12. Conclusion and Recommendations for Future Work

12.1 Conclusion

The Smart Kitchen project successfully demonstrates the required FreeRTOS concepts in an integrated embedded application. Four concurrent tasks provide lighting control, oven control, user override handling and UART logging. Equal-priority lighting and oven tasks support round-robin time slicing, while a higher-priority override task provides fast response to human input. FreeRTOS queues, mutexes and a counting semaphore coordinate communication and protect shared state and UART access. The oven subsystem adds fail-safe behavior for invalid sensing and over-temperature conditions, proving that RTOS synchronization is being applied to a realistic safety problem rather than only to isolated laboratory examples.

12.2 Recommendations for Future Work

- Replace the PE5 mock light input with a real LDR voltage divider on PE3/AIN0 and calibrate the threshold from measured lux conditions.
- Replace the MCU internal temperature sensor with an external oven-rated thermal sensor and select realistic temperature setpoints.
- Add a fourth independent physical output such as a buzzer or dedicated fault LED for unambiguous alarms.
- Move switch debounce out of the interrupt handlers into a non-blocking software timer or task-based debounce state machine.
- Add a watchdog timer and safe recovery policy for unexpected task/kernel failure.
- Use event groups or direct-to-task notifications where appropriate to compare memory and timing efficiency against the current queue/semaphore design.
- Add persistent fault/event statistics and timestamped logs.
- Measure actual task execution time, stack high-water marks and CPU utilization to perform a formal schedulability/resource analysis.

References

- [1] Ain Shams University, Faculty of Engineering, CSE 323/411: Advanced Embedded Systems - Major Course Assignment (Smart Kitchen System), Summer Semester 2026.
- [2] FreeRTOS / Amazon Web Services, FreeRTOS Kernel documentation: task management, queues, semaphores, mutexes and Cortex-M interrupt integration.
- [3] Texas Instruments, TM4C123GH6PM Microcontroller Data Sheet, including ADC, GPIO, UART and internal temperature-sensor documentation.
- [4] Texas Instruments, Tiva C Series TM4C123G LaunchPad Evaluation Kit User Guide.
- [5] Arm / Keil, CMSIS-FreeRTOS software component pack, version 11.3.0, as referenced by the submitted Keil project configuration.
- [6] Submitted project source archive: main.c, tasks.c/h, adc.c/h, led.c/h, switch.c/h, uart.c/h and RTE/RTOS/FreeRTOSConfig.h.